Week 9.2 Excercise - mlflow_observability_recreation

May 14, 2026

1 Recreating Listing 11.4: MLflow Observability for a Simple GenAI App

This notebook recreates the Chapter 11 MLflow observability example in a notebook-first workflow.

1.1 What the original author is doing

1. Configures OpenAI model settings and an MLflow tracking URI.
2. Defines helper functions to count tokens and generate responses.
3. Logs runtime metrics to MLflow for each model request (latency, tokens, request count).
4. Runs a very simple chat-style app loop and tracks everything in one MLflow run.

1.2 My evaluation

The design is intentionally minimal, which is good for learning observability basics. The core value is that the app logs both performance (latency) and usage (token counts) on every call, so you can inspect trends over time in MLflow. For production, I would add stronger error handling, richer trace IDs, and token/cost logging by request and session.

Running a Simple Generative AI Application with MLflow Observability

Author: Vijay Sharma
Institution: Bellevue University Course: DSC670-T301 Advanced Uses of Generative AI (2265-1)
Instructor: Frank Neugebauer
Date: May 13, 2026

This notebook recreates the MLflow observability workflow presented in Chapter 11, Listing 11.4, and demonstrates how a lightweight generative AI application can be instrumented for monitoring. The implementation follows the author's structure by configuring model settings, issuing chat completion requests, and logging core telemetry such as request latency and token usage to MLflow. In addition, the notebook includes explanatory commentary and a brief evaluation of design choices, strengths, and production considerations. The objective is to show that even a minimal application can produce meaningful operational visibility when observability is integrated from the beginning.

[ ]:
```
%pip install -q mlflow tiktoken prometheus-client openai colorama
```

1.3 MLflow Tracking — No Server Required

This notebook uses a **local file-based** tracking URI (`./mlruns`), so no separate `mlflow server` process is needed.

To view results in the MLflow UI after running the notebook, run this in a terminal:

`mlflow ui --backend-store-uri ./mlruns`

Then open http://127.0.0.1:5000.

[2]:
```
import os
import time
import uuid
import mlflow
import tiktoken as tk
from openai import OpenAI
from prometheus_client import Counter, Histogram, start_http_server
from colorama import Fore, Style, init

# Visual formatting for the tiny console app
init(autoreset=True)

# OpenAI and app config (matching the listing style)
API_KEY =␣
 ↪"sk-proj-ruj4TM7gtrhhh3h4ocnXrN_jN5cMl1Ai33Ol7Kq_X8priylWKHdawHKVxVsD-s3EEwoBta6IluT3BlbkFJ
MODEL = "gpt-4o-mini"
# Use local file storage - no running server required
MLFLOW_URI = os.path.join(os.getcwd(), "mlruns")
EXPERIMENT_NAME = "GenAI_book"
TEMPERATURE = 0.7
TOP_P = 1
FREQUENCY_PENALTY = 0
PRESENCE_PENALTY = 0
MAX_TOKENS = 300
DEBUG = False

if not API_KEY:
    raise ValueError(
        "Set OPENAI_API_BOOK_KEY or OPENAI_API_KEY before running this notebook.
 ↪"
    )

client = OpenAI(api_key=API_KEY)

# MLflow tracking setup - file:// URI works without a running server
mlflow.set_tracking_uri(f"file:///{MLFLOW_URI}")
mlflow.set_experiment(EXPERIMENT_NAME)

print(f"MLflow tracking to: {MLFLOW_URI}")
print("Configured OpenAI client and MLflow tracking.")
```

```
/opt/anaconda3/lib/python3.12/site-
packages/mlflow/tracking/_tracking_service/utils.py:184: FutureWarning: The
```

```
filesystem tracking backend (e.g., './mlruns') is deprecated as of February
2026. Consider transitioning to a database backend (e.g., 'sqlite:///mlflow.db')
to take advantage of the latest MLflow features. See
https://mlflow.org/docs/latest/self-hosting/migrate-from-file-store for
migration guidance.
  return FileStore(store_uri, store_uri)

MLflow tracking to: /Users/vijsharm/Downloads/Bellevue
University/DSC670Assignment/mlruns
Configured OpenAI client and MLflow tracking.


Configured OpenAI client and MLflow tracking.
```

[3]:
```python
# Optional Prometheus metrics (the listing references prometheus_client)
REQUEST_COUNTER = Counter(
    "genai_request_count",
    "Total number of requests sent to the LLM"
)
REQUEST_LATENCY = Histogram(
    "genai_request_latency_seconds",
    "End-to-end LLM request latency in seconds"
)

PROM_PORT = 8001
try:
    start_http_server(PROM_PORT)
    print(f"Prometheus metrics available at http://127.0.0.1:{PROM_PORT}/
 ↪metrics")
except OSError:
    print("Prometheus endpoint already running. Continuing without restarting␣
 ↪it.")
```

```
Prometheus metrics available at http://127.0.0.1:8001/metrics
```

[4]:
```python
def print_user_input(text: str) -> None:
    print(f"{Fore.GREEN}You:{Style.RESET_ALL} {text}")

def print_ai_output(text: str) -> None:
    print(f"{Fore.BLUE}AI Assistant:{Style.RESET_ALL} {text}")

def count_tokens(text: str, encoding_name: str = "cl100k_base") -> int:
    """Count tokens with tiktoken."""
    encoding = tk.get_encoding(encoding_name)
    return len(encoding.encode(text, disallowed_special=()))

def generate_text(conversation: list, max_tokens: int = 100) -> str:
    """Core function from the listing pattern: call model, time it, log metrics.
 ↪"""
```

```
    start_time = time.time()
    response = client.chat.completions.create(
        model=MODEL,
        messages=conversation,
        temperature=TEMPERATURE,
        max_tokens=max_tokens,
        top_p=TOP_P,
        frequency_penalty=FREQUENCY_PENALTY,
        presence_penalty=PRESENCE_PENALTY
    )
    latency = time.time() - start_time

    message_response = response.choices[0].message.content or ""

    prompt_tokens = count_tokens(conversation[-1]["content"])
    conversation_tokens = count_tokens(str(conversation))
    completion_tokens = count_tokens(message_response)

    # MLflow observability logs
    mlflow.log_metrics({
        "request_count": 1,
        "request_latency": latency,
        "prompt_tokens": prompt_tokens,
        "completion_tokens": completion_tokens,
        "conversation_tokens": conversation_tokens
    })

    mlflow.log_params({
        "model": MODEL,
        "temperature": TEMPERATURE,
        "top_p": TOP_P,
        "frequency_penalty": FREQUENCY_PENALTY,
        "presence_penalty": PRESENCE_PENALTY
    })

    # Optional Prometheus observability
    REQUEST_COUNTER.inc()
    REQUEST_LATENCY.observe(latency)

    if DEBUG:
        run = mlflow.active_run()
        print(f"Run ID: {run.info.run_id if run else 'None'}")

    return message_response
```

1.4 Run the Simple App (Notebook version)

The book uses a terminal `while True` chat loop. In a notebook, a short fixed list of prompts is easier to reproduce and grade while still showing the same observability behavior.

[ ]:
```
# Equivalent to the listing's app flow, but with scripted prompts for notebook execution
mlflow.autolog()

conversation = [{"role": "system", "content": "You are a helpful assistant."}]
demo_prompts = [
    "Summarize why observability matters in GenAI apps.",
    "Give me three metrics I should track for latency and token use."
]

with mlflow.start_run(run_name=f"listing_11_4_notebook_{uuid.uuid4().hex[:8]}") as run:
    print(f"MLflow Run ID: {run.info.run_id}")
    for user_input in demo_prompts:
        print_user_input(user_input)
        conversation.append({"role": "user", "content": user_input})

        ai_output = generate_text(conversation, MAX_TOKENS)
        print_ai_output(ai_output)

        conversation.append({"role": "assistant", "content": ai_output})

    # Helpful artifact for reproducibility
    transcript = "\n\n".join([f"{m['role']}: {m['content']}" for m in conversation])
    mlflow.log_text(transcript, "conversation_transcript.txt")

print("Done. Open MLflow UI to inspect metrics and params for this run.")
```

```
2026/05/14 15:45:36 INFO mlflow.tracking.fluent: Autologging successfully
enabled for openai.
```

1.5 Optional: Interactive Loop (closest to the book script)

Uncomment and run this if you want the terminal-style interaction inside the notebook.

```
with mlflow.start_run(run_name="interactive_notebook_loop"):
    conversation = [{"role": "system", "content": "You are a helpful assistant."}]
    while True:
        user_input = input("User: ")
        if user_input.lower() in ["exit", "quit", "q", "e"]:
            break
        conversation.append({"role": "user", "content": user_input})
        ai_output = generate_text(conversation, MAX_TOKENS)
```

```
print_ai_output(ai_output)
conversation.append({"role": "assistant", "content": ai_output})
```

1.6 Commentary and Evaluation

This implementation preserves the purpose of Listing 11.4: make observability visible with minimal app complexity. The author's central idea is to treat every model response as an observable event. By logging latency and token usage each time, you can answer practical questions quickly: Is response time stable? Are token counts drifting up? Is one prompt pattern consistently expensive?

I think this is a strong teaching example because it demonstrates an immediate monitoring habit instead of adding telemetry after problems appear. I also like that the code logs both metrics and configuration parameters, which improves reproducibility. The biggest limitation is that it is still a single-process demo. In real deployments, I would add request IDs, user/session tags, explicit error metrics, and cost estimates per call. I would also track guardrail outcomes and model refusal rates so safety and quality can be reviewed alongside performance.